

Desarrollo Arquitectura de Software de acuerdo con el Patrón de Diseño Seleccionado

Lucy Estefany Izquierdo Jaramillo

Alejandro Sepúlveda Duarte

Centro de Comercio Regional Antioquia

Servicio Nacional de Aprendizaje-SENA

Tecnología en Análisis y Desarrollo de Software

José Ignacio Botero Osorio

27 de septiembre del 2025

Versión actualizada: 24 de julio de 2026

Contenido

Introducción	3
Objetivos	4
Objetivo general.....	4
Objetivo específico	4
Patrón de diseño seleccionado	5
Justificación	5
Vista de Componentes.....	5
Vista de despliegue del software.....	7
Herramientas necesarias para optimizar los procesos.....	9
Conclusión	11
Referencias Bibliográficas	12

Introducción

El desarrollo de software requiere de una arquitectura sólida que garantice la calidad, mantenibilidad y escalabilidad de la solución informática. La arquitectura de software se compone de un conjunto de patrones y abstracciones que permiten estructurar el sistema de forma organizada, facilitando la interacción entre los diferentes módulos y el código fuente.

En este trabajo se presenta la construcción de la arquitectura de software basada en un patrón de diseño seleccionado, con el fin de aplicar buenas prácticas de codificación y optimizar los procesos de desarrollo. Para ello, se incorporan las vistas de componentes y de despliegue, que permiten visualizar tanto la organización interna del software como las condiciones necesarias para su implantación en un entorno real. Asimismo, se seleccionan y justifican las herramientas que apoyan la implementación, modelado y mantenimiento de la solución propuesta.

Objetivos

Objetivo general

Diseñar la arquitectura de software del sistema de inventario para PYMES, incorporando patrones de diseño que garanticen buenas prácticas de desarrollo, mantenibilidad y escalabilidad, mediante la elaboración de la vista de componentes, la vista de despliegue y la selección de herramientas adecuadas para su implementación.

Objetivo específico

- Seleccionar e implementar un patrón de diseño apropiado para la solución informática, de acuerdo con los requerimientos del sistema.
- Elaborar la vista de componentes que permita representar la organización lógica del software en sus diferentes módulos.
- Diseñar la vista de despliegue para definir las condiciones físicas y tecnológicas necesarias en la implantación del sistema.
- Identificar y justificar las herramientas tecnológicas que optimicen los procesos de desarrollo, modelado y gestión del software.

Patrón de diseño seleccionado

Para el sistema de inventario de PYMES se propone utilizar el patrón de diseño **MVC (Modelo – Vista – Controlador)**, aplicado sobre una arquitectura cliente-servidor con API REST:

- **Modelo:** los modelos de Django (usuarios, productos, inventario, proveedores, clientes, compras, ventas) gestionan los datos, las reglas de negocio y la interacción con PostgreSQL.
- **Vista:** la aplicación Angular (SPA), compilada y servida al navegador, es la que renderiza la interfaz gráfica (formularios, tablas, dashboards); no es el servidor quien genera el HTML de las vistas, como ocurriría en un MVC clásico monolítico.
- **Controlador:** las vistas de Django (views.py de cada app, expuestas mediante Django REST Framework) actúan como controladores: reciben las peticiones HTTP de Angular, aplican las reglas de permisos por rol y devuelven las respuestas en formato JSON.

Justificación

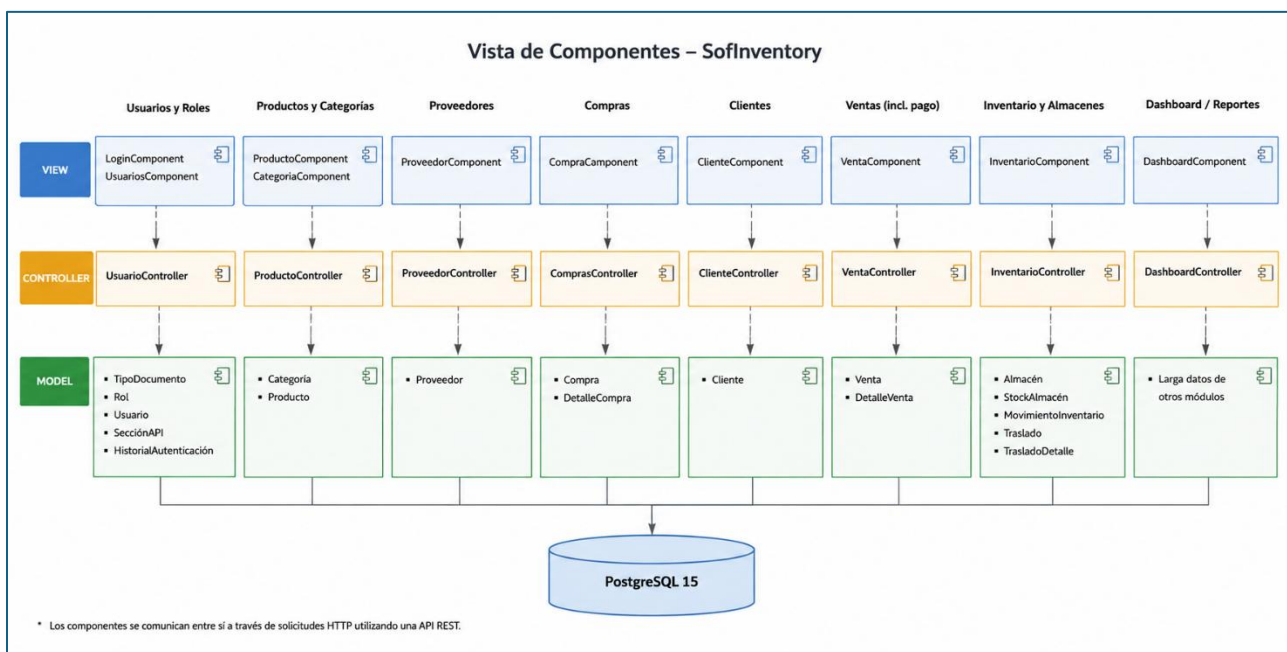
El patrón MVC, adaptado a esta arquitectura cliente-servidor desacoplada, sigue siendo apropiado para un sistema de inventario como SofInventory: permite adaptar la solución a distintos tipos de negocio (ferreterías, papelerías, farmacias) ajustando principalmente la Vista (Angular) o ampliando la lógica del Modelo (Django), sin reescribir toda la arquitectura. La separación adicional entre frontend y backend, cada uno desplegado de forma independiente, aporta beneficios que el MVC monolítico original no contemplaba: permite escalar el frontend y el backend por separado, y permite que otros clientes (una futura app móvil, por ejemplo) reutilicen la misma API REST sin duplicar lógica de negocio.

Vista de Componentes

La versión original agrupaba el sistema en siete módulos, pero el diagrama solo representaba cuatro controladores y omitía varias entidades reales, a la vez que incluía entidades (Serial, Lote, MetodoPago como tabla) que nunca se implementaron. La vista corregida refleja las ocho aplicaciones (apps) reales del backend Django, cada una con su Vista (página Angular), su Controlador (views.py) y su Modelo (tablas en PostgreSQL):

- Usuarios y Roles
- Productos y Categorías
- Proveedores
- Compras
- Clientes
- Ventas (los datos de pago viven dentro de Venta, no en un módulo de Pagos independiente)
- Inventario y Almacenes
- Dashboard / Reportes (agrega datos de los demás módulos, no tiene tablas propias) Base de Datos

Figura 1.
Vista de componentes



Nota. Elaboración propia realizada.

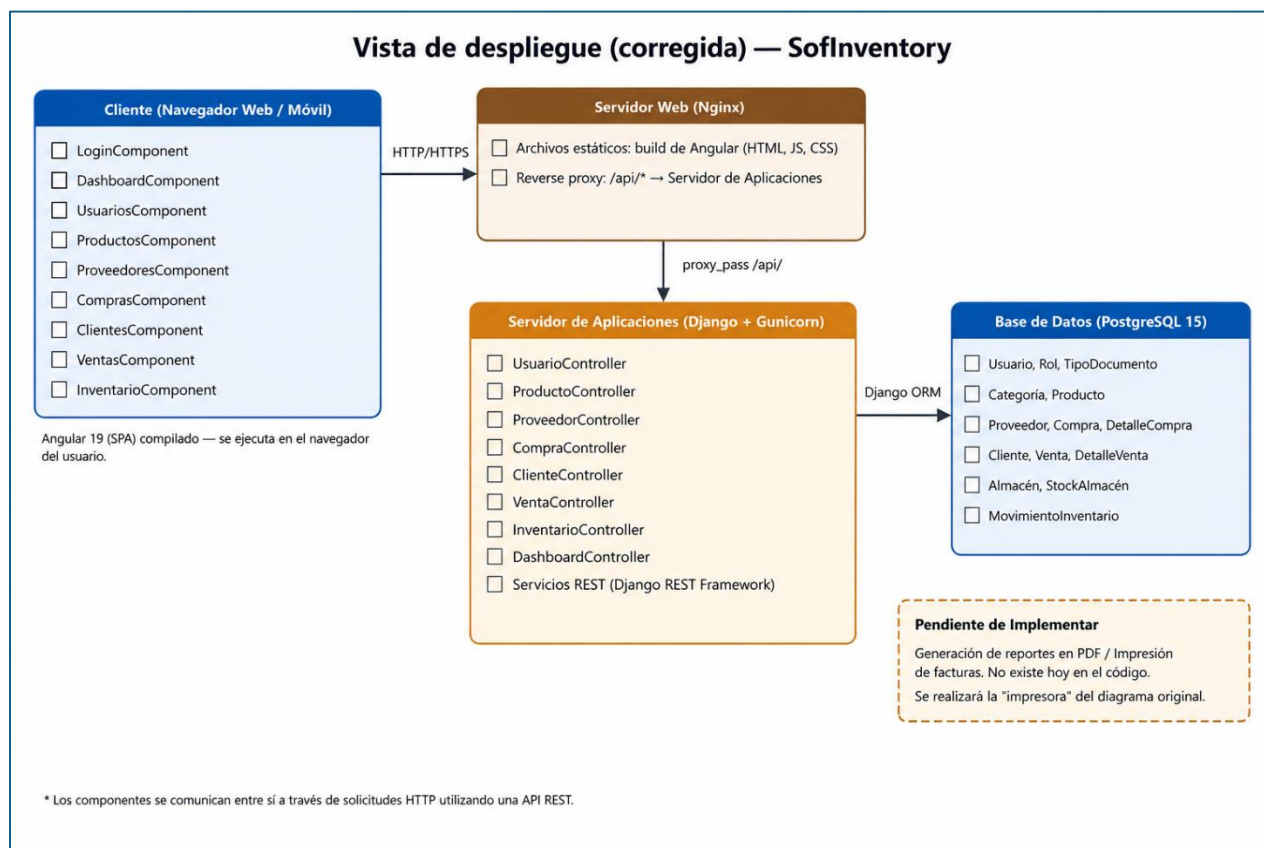
Tal como se muestra en la **figura 1**, la vista de componentes refleja la arquitectura MVC aplicada al sistema de inventario y ventas, con una clara separación de responsabilidades. Las vistas se comunican con sus respectivos controladores, quienes a su vez interactúan con el modelo de datos. Esta estructura permite aplicar principios de la programación orientada a objetos como la abstracción, el desacoplamiento y la escalabilidad, facilitando el mantenimiento y evolución del sistema.

Vista de despliegue del software

El despliegue en docker-compose.yml, usa tres contenedores en el entorno de desarrollo: uno para la base de datos (PostgreSQL), uno para el backend (Django + Gunicorn) y uno para el frontend (Nginx sirviendo el build de Angular). En el entorno de producción actual (Render), el propio backend Django sirve el build de Angular desde un único servicio, con PostgreSQL como

base de datos administrada aparte; por eso la vista de despliegue distingue el rol lógico de cada componente, que puede o no coincidir con un contenedor físico separado según el entorno.

Figura 2.
Vista de despliegue del software



Nota. Elaboración propia realizada.

Tal como se muestra en la **figura 2**, la vista de despliegue corregida refleja que los clientes acceden al sistema a través del navegador, el cual solicita la interfaz al servidor web y, a través de este, se comunica con el servidor de aplicaciones mediante la API REST. El servidor de aplicaciones concentra toda la lógica de negocio y controladores, y es el único componente que se comunica directamente con la base de datos PostgreSQL. Este esquema es coherente con un despliegue en contenedores Docker y es igualmente válido para un entorno de producción en la nube.

Herramientas necesarias para optimizar los procesos

Con el fin de garantizar la calidad, mantenibilidad y escalabilidad del sistema de inventario y ventas, se seleccionan un conjunto de herramientas que apoyan tanto el desarrollo como el despliegue y la gestión del proyecto:

1. Lenguaje y framework de desarrollo

- Backend: Python con Django y Django REST Framework (decisión ya tomada e implementada, no una alternativa entre varias).
- Frontend: Angular 19 (decisión ya tomada e implementada; las alternativas de la versión original — React, Spring Boot, .NET — ya no aplican al proyecto actual).

2. Base de datos: PostgreSQL 15 (decisión ya tomada; MySQL, mencionado como alternativa en la versión original, no se usa).

3. Entorno de desarrollo: Visual Studio Code, usado de forma consistente en el equipo para backend y frontend.

4. Control de versiones: Git con repositorio remoto en GitHub.

5. Metodología de trabajo: Se mantiene la recomendación de Scrum o Kanban con Trello/Jira; esta parte no se verifica contra el código y sigue siendo una recomendación de proceso, no una herramienta técnica del sistema.

6. Pruebas y calidad (estado real vs. recomendado)

- **Estado real:** el proyecto solo cuenta con scripts de prueba manuales ad-hoc (por ejemplo `test_proveedores_api.py`), que ejecutan la API directamente sin usar un framework formal de pruebas.

- **Recomendado:** migrar estos scripts a pruebas con Django TestCase o Pytest, y considerar Selenium u otra herramienta de pruebas end-to-end para el frontend Angular, tal como planteaba la versión original.

7. Despliegue y automatización

- **Docker:** en uso real, con Dockerfiles independientes para backend y frontend, y docker-compose.yml para el entorno local.
- **CI/CD:** sigue siendo una recomendación pendiente de adoptar, no una herramienta ya implementada.

Conclusión

El desarrollo de la arquitectura de software para el sistema de inventario y ventas permitió evidenciar la importancia de aplicar buenas prácticas en el diseño y la planificación de soluciones informáticas. A través de la vista de componentes se representó la aplicación del patrón MVC, que facilita la separación de responsabilidades y fortalece la mantenibilidad del sistema. Asimismo, la vista de despliegue permitió identificar las condiciones de implantación de la solución, tanto en un entorno local como con proyección hacia entornos en línea. Finalmente, la selección de herramientas como Python con Django, PostgreSQL, Git y Docker, entre otras, garantiza un proceso de desarrollo más ágil, seguro y escalable. De esta manera, se sientan las bases para un sistema robusto, adaptable y alineado con las necesidades actuales y futuras de la organización.

Referencias Bibliográficas

LMS ZAJUNA. (s.f.). “Diseño de patrones de software”. Material de formación.

<https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118/Contenido/OVA/CF18/index.html#/>

nicosiored. (2018, 22 marzo). *Diagrama de Despliegue - 22 - Tutorial UML en español* [Video].

YouTube. <https://www.youtube.com/watch?v=NSB0ATJUavA>

nicosiored. (2018a, marzo 8). *Diagrama de Componentes I - 20- Tutorial UML en español* [Video]. YouTube. https://www.youtube.com/watch?v=oOycG_n1ARs